

Машинное обучение с использованием нейронных сетей

Оглавление

Введение.....	2
Классификация ирисов	2
Нейроны и перцептроны	3
Нейроны.....	3
Перцептроны	4
Улучшение модели.....	5
Веса.....	5
Смещения.....	6
Функции активации	7
Функция выпрямленного линейного блока (ReLU).....	7
Логистическая (сигмоидная) функция	8
Функция softmax.....	9
Слои и глубокое обучение	9
Слои.....	9
Глубокое обучение	10
Обучение.....	11
Инициализация весов	12
Вычисление функции потерь.....	12
Прогнозирование и среднеквадратическая ошибка (MSE).....	13
Классификация и метод максимального правдоподобия.....	13
Оптимизация методом градиентного спуска	14
Мини-пакетная обработка	16
Пример применения — классификация ирисов	16
Обучение сети.....	18
Обработка слоев с помощью мини-пакетов.....	18
Вычисление градиента функции потерь — Часть 1.....	19
Вычисление градиента функции потерь — Часть 2.....	22
Тестирование сети.....	24
Задание для самостоятельной работы	26

Введение

Важность машинного обучения в современной обработке данных невозможно переоценить, и её значение растёт с каждым днём. Рассмотрим применение вычислительных шейдеров WebGPU для машинного обучения в веб-приложении. Наиболее популярной реализацией машинного обучения являются нейронные сети.

Будем разрабатывать приложение WebGPU для реализации нейронной сети классификации ирисов. Представленная методология может быть применена и к другим задачам.

Классификация ирисов

Распространенный тест для нейронных сетей включает классификацию ирисов на основе четырех физических характеристик: ширины чашелистика, длины чашелистика, ширины лепестка и длины лепестка.

Цель этой главы — создать нейронную сеть, которая принимает эти четыре характеристики и определяет, к какому виду принадлежит ирис. В частности, сеть будет выдавать три значения от 0,0 до 1,0:

- оценочная вероятность того, что ирис принадлежит к виду *setosa*
- оценочная вероятность того, что ирис принадлежит к виду *versicolor*
- оценочная вероятность того, что ирис принадлежит к виду *virginica*

На рис. 1 показаны результаты шести тестов, проведенных разработанным приложением. В каждом случае три значения, вычисленные приложением, сравниваются с фактическими результатами. Приложение никогда не бывает идеальным, но в каждом тесте максимальная вычисленная вероятность соответствует фактической классификации. Это означает, что нейронная сеть правильно классифицировала сорта ирисов.

Actual classification: 1.000 0.000 0.000

Computed outputs: 0.996 0.004 0.000

Result: SUCCESS

Actual classification: 1.000 0.000 0.000

Computed outputs: 0.993 0.007 0.000

Result: SUCCESS

Actual classification: 0.000 0.000 1.000

Computed outputs: 0.000 0.034 0.966

Result: SUCCESS

Actual classification: 0.000 1.000 0.000

Computed outputs: 0.004 0.978 0.018

Result: SUCCESS

Actual classification: 0.000 1.000 0.000

Computed outputs: 0.010 0.989 0.001

Result: SUCCESS

Actual classification: 0.000 1.000 0.000

Computed outputs: 0.003 0.657 0.340

Result: SUCCESS

Рис. 1. Результаты классификации сортов ирисов.

Нейроны и перцептроны

Нейроны

В начале XIX века хирурги внимательно изучили клетки, составляющие нервную ткань. Они называли эти нервные клетки *нейронами*, и на рисунке 2 представлено упрощенное изображение их структуры.

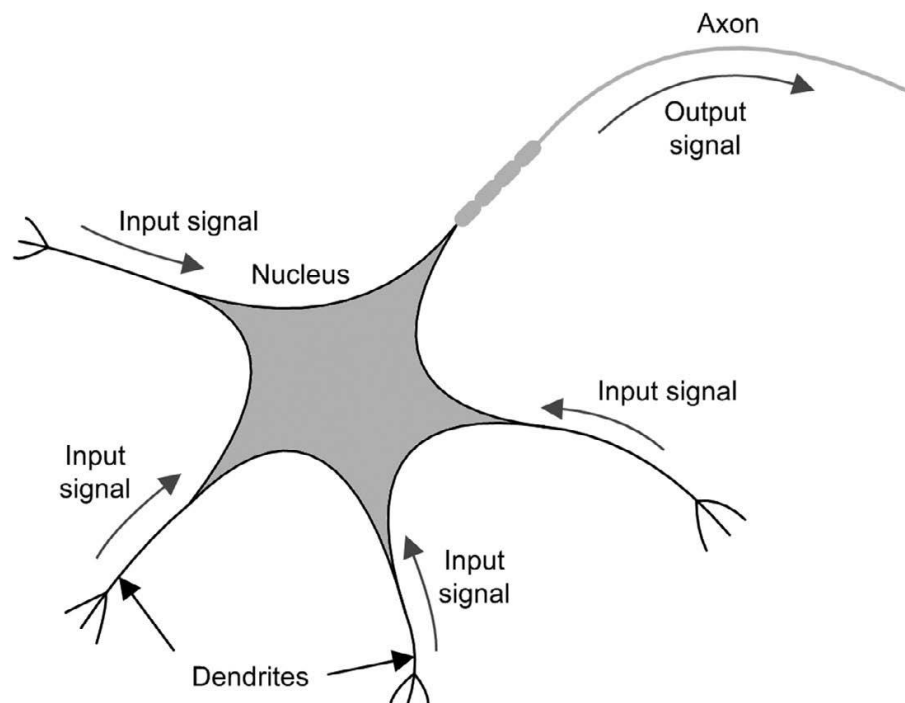


Рис. 2. Упрощенная структура нейрона.

Нейрон получает электрическую стимуляцию через свои дендриты. Химические вещества в клетке накапливают поступающее электричество, и по мере увеличения силы поступающих сигналов внутреннее напряжение нейрона возрастает.

Когда напряжение нейрона превышает пороговое значение, нейрон посылает сигнал через свой *аксон*, показанный в правом верхнем углу. Этот выходной сигнал может стимулировать другие нейроны, а это значит, что срабатывание одного нейрона может вызвать срабатывание целой серии других нейронов.

Наука о неврологии продвинулась семимильными шагами с XIX века, и сегодня мы знаем, что нейроны делают гораздо больше, чем просто передают сигналы от одной клетки к другой. Но сейчас нам нужно усвоить всего два момента:

1. Нейрон получает множество входных сигналов и производит один выходной сигнал. Этот выходной сигнал может служить входным сигналом для другого нейрона.
2. У каждого нейрона есть пороговое значение, и нейрон будет производить выходной сигнал только в том случае, если его внутреннее напряжение превышает пороговое значение.

Теперь рассмотрим *перцептроны*, которые представляют собой схемы, работа которых напоминает работу нейронов.

Перцептроны

В 1960-х годах исследователь по имени Фрэнк Розенблатт сконструировал схему, которая вела себя как нейрон. Он назвал его *перцептроном*, и рисунок 3 дает представление о том, как он работает.

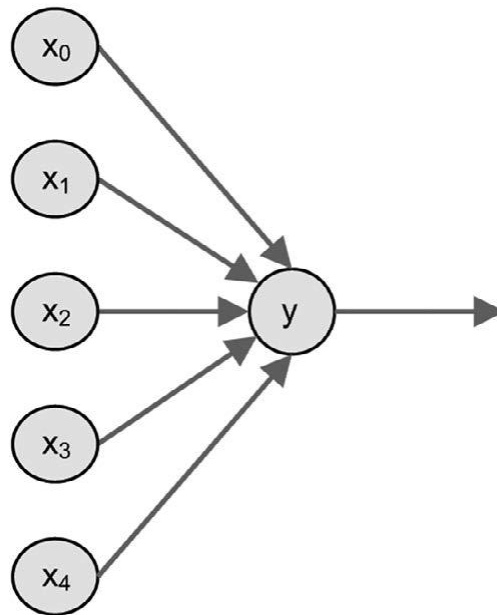


Рис 3. Аппроксимация нейронного поведения с помощью перцептрона.

Перцептрон получает несколько входных сигналов и выдает выходной сигнал в виде числовых значений. На рисунке эти входные значения обозначены как x_0 – x_4 .

Если сумма входных значений превышает пороговое значение, выходной сигнал будет равен 1. Если сумма сигналов не достигает порога, выходной сигнал будет равен 0.

Например, предположим, что x_0 равно 1.5, x_1 равно 0.5, x_2 равно –1.0, x_3 равно 0.5 и x_4 равно 1.0. В этом случае сумма значений равна 2.5. Если пороговое значение равно 2.0, перцептрон выдаст выходной сигнал 1, потому что сумма больше порога. Если пороговое значение равно 3.0, перцептрон выдаст выходной сигнал 0, потому что сумма ниже порога.

Обозначив входные данные как x_i , а выходные как y , выход перцептрона можно выразить следующим уравнением:

$$y = \frac{1, \text{ если } \sum_i x_i > threshold}{0, \text{ в противном случае}}$$

Подобно нейронам, перцептроны можно соединять таким образом, чтобы выход одного служил входом для другого. Рис. 4 дает представление о том, как это выглядит. Перцептроны могут иметь разное количество входных значений, но каждый всегда выдает один выход.

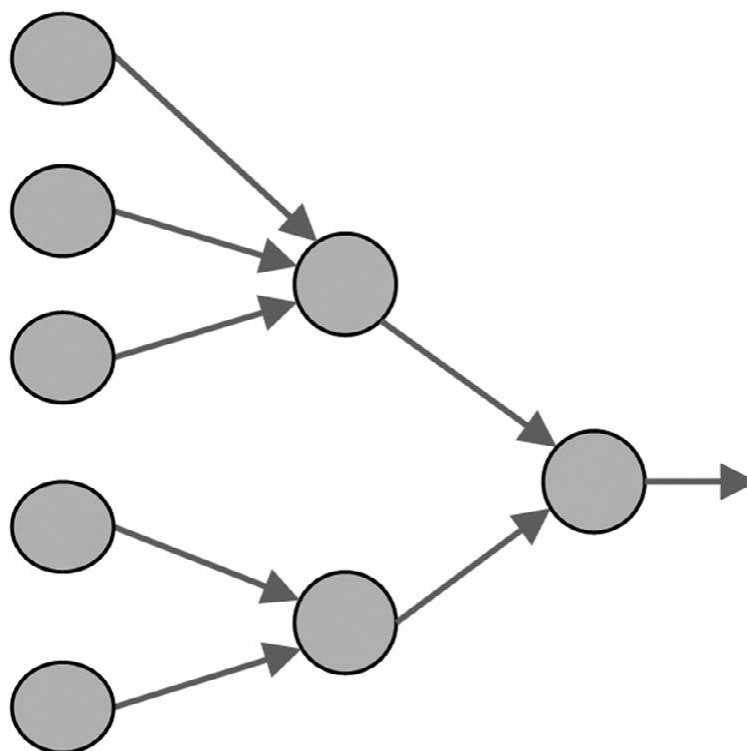


Рис. 4. Соединение перцептронов.

Перцептроны полезны при изучении предмета, но они не используются в практическом машинном обучении. Это связано с тем, что поведение перцептрона *статично* — его работу нельзя обновить. Процесс обновления поведения схемы называется *обучением*, и это одна из главных проблем (возможно, *центральная проблема*) современного машинного обучения.

Улучшение модели

После того, как Розенблатт продемонстрировал возможности своего перцептрона, исследователи обновили его модель несколькими способами. Перечислим три основных улучшения:

1. Каждый входной сигнал умножается на значение, называемое *весом*.
2. Вместо сравнения суммы входных сигналов с пороговым значением добавляется дополнительный входной сигнал, называемый *смещением*.
3. Сумма взвешенных входных сигналов (включая смещение) передается на *функцию активации*, которая определяет выходной сигнал.

В результате этих изменений перцептроны становятся ближе к практическому применению. По этой причине мы перестанем использовать термин «*перцептрон*» и начнем называть эти элементы *узлами*.

Веса

На рис. 2 каждый входной сигнал оказывает одинаковое влияние на определение выходного сигнала. Но в реальном машинном обучении некоторые входные сигналы оказывают большее влияние, чем другие, на процесс принятия решений. Кроме того, некоторые сигналы могут оказывать отрицательное влияние на результат.

Чтобы отразить это, каждый входной сигнал умножается на значение, называемое *весом*. На рис. 5 показано, как выглядит узел со взвешенными входными сигналами.

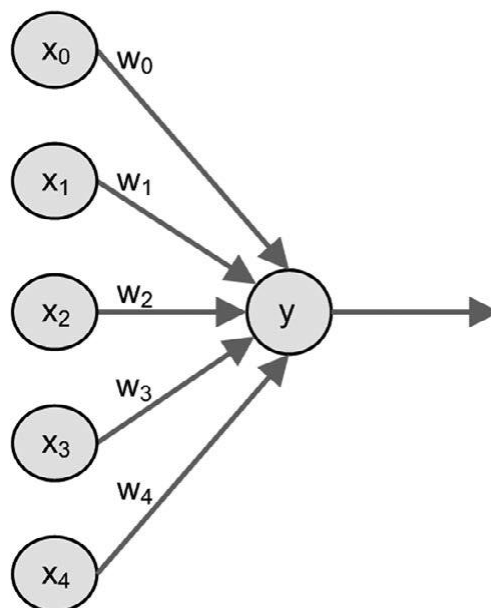


Рис. 5. Узлы с весами.

Для вычисления выходного сигнала каждое входное значение умножается на свой вес, и произведения суммируются. Затем сумма сравнивается с порогом узла. Узел выдает 1, если сумма больше порога, и 0 в противном случае.

Обозначив входные данные как x_i , а веса как w_i , работу узла можно выразить следующим уравнением:

$$y = \frac{1, \text{ если } \sum_i w_i x_i > \text{threshold}}{0, \text{ в противном случае}}$$

Рассмотрим пример. Предположим, что входные данные x_i равны $\{5.3, -2.0, 1.7, -0.3\}$, а их веса w_i равны $\{0.6, -0.2, 1.4, 2.3\}$. Сумма взвешенных входных данных может быть вычислена следующим образом:

$$\begin{aligned} \sum_i x_i w_i &= (5.3 \cdot 0.6) + (-2.0 \cdot -0.2) + (1.7 \cdot 1.4) + (-0.3 \cdot 2.3) = \\ &= 3.18 + 0.4 + 2.38 - 0.69 = 5.27 \end{aligned}$$

Сумма взвешенных входных данных равна 5.27, поэтому, если пороговое значение узла равно 5.0, он выдаст на выходе 1. Если пороговое значение равно 6.0, узел выдаст 0.

Веса играют важную роль в машинном обучении, поскольку они позволяют обновлять работу нейронной сети. Процесс обновления этих весов называется *обучением*.

Смещения

Вместо того чтобы отслеживать пороговые значения, обычно упрощают модель, добавляя входное значение с постоянным значением, называемое *смещением*. На рис. 6 показано, как это выглядит.

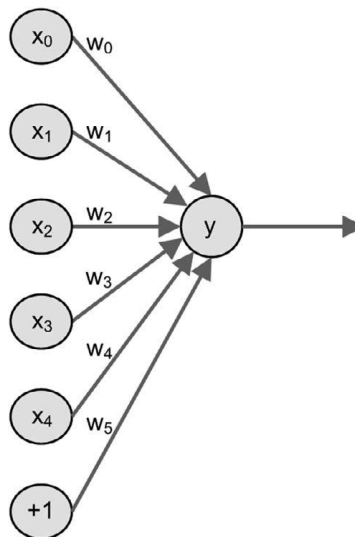


Рис. 6. Добавление входного сигнала смещения.

Смещение получает вес так же, как и любой другой вход. По этой причине имеет смысл установить значение входного смещения равным 1. Именно поэтому нижний узел слева имеет значение +1. По сути, вес смещения служит пороговым значением узла.

Функции активации

Единичная ступенчатая функция $u(x)$ возвращает 1, если x больше или равно 0, и 0, если x меньше 0. Используя эту функцию, связь между выходом узла y и его взвешенными входами $w_i x_i$ может быть выражена следующим уравнением:

$$y = u \left(\sum_i w_i x_i \right)$$

Единичная ступенчатая функция проста для понимания, но на практике специалисты по обработке данных используют более подходящие функции для получения выходного сигнала узла. Они называются *функциями активации*. Было разработано множество типов функций активации, и все они обладают тремя важными характеристиками:

1. *нелинейность* — реальные задачи машинного обучения являются нелинейными
2. *монотонное возрастание* — если $x > y$, $f(x)$ должно быть больше или равно $f(y)$
3. *дифференцируемость* — производная функции должна существовать и быть легко вычисляемой

Рассмотрим три популярные функции активации: функция выпрямленного линейного блока (ReLU), логистическая (сигмоидная) функция и функция softmax.

Функция выпрямленного линейного блока (ReLU)

В электрических цепях выпрямитель принимает входной сигнал и передает выходной сигнал, равный входному, если входной сигнал положительный. Если входной сигнал неположительный, выпрямитель выдает на выходе ноль.

Функция выпрямленного линейного блока, или ReLU, выполняет аналогичную операцию. Ее выход равен входному сигналу, если входной сигнал положительный, и равен 0 в противном случае. Другими словами, функция ReLU возвращает максимум из входного значения и 0. В коде это можно выразить как $\max(0, x)$. На рис. 7 показан график функции.

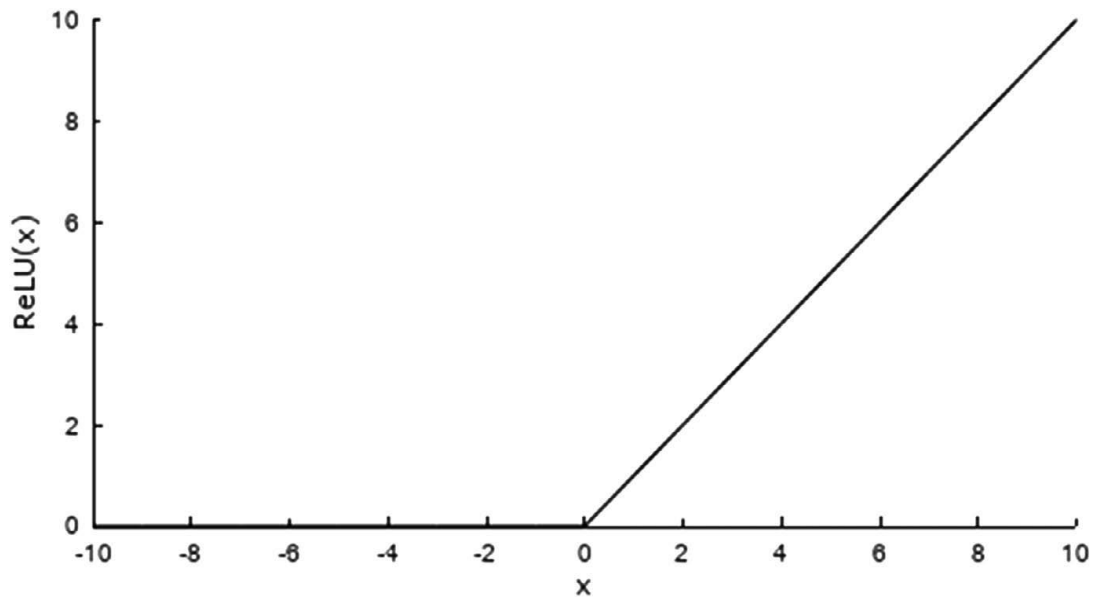


Рис. 7. Функция активации выпрямленного линейного блока (ReLU).

Функция ReLU популярна в приложениях машинного обучения, но у нее есть один существенный недостаток: она разрывна в точке 0. По этой причине некоторые специалисты по обработке данных предпочитают функцию экспоненциального линейного блока (ELU). Как и ReLU, функция ELU возвращает входной сигнал, если входной сигнал больше 0. В отличие от ReLU, ELU возвращает экспоненту от входного сигнала минус 1, $\exp(x) - 1$, если входной сигнал меньше 0. Это гарантирует, что выход узла всегда непрерывен.

Логистическая (сигмоидная) функция

Логистическая функция используется в приложениях, которым необходимо классифицировать признаки по одной из двух категорий. Математически её можно выразить следующим уравнением:

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

На рис. 8 показан график логистической функции для значений x от -8 до 8.

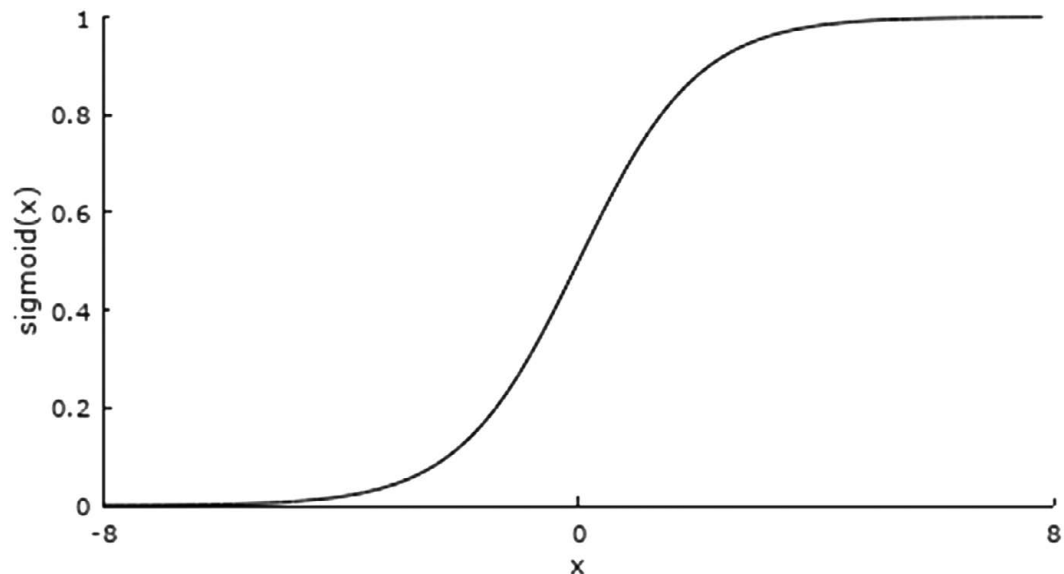


Рис. 8. Логистическая (сигмоидная) функция активации.

Логистическая функция имеет S-образную форму, и поскольку сигма (σ) — это греческая буква, обозначающая S, её обычно называют сигмоидной функцией или $\sigma(x)$. Она обладает тремя свойствами, которые делают её идеальной для классификации данных:

1. Её выходное значение находится в диапазоне от 0 до 1, что означает, что результат узла можно интерпретировать как вероятность.
2. $\sigma(0) = 0,5$, что означает, что точка данных в центре с одинаковой вероятностью может принадлежать к обеим категориям.
3. Функция симметрична относительно оси y : $\sigma(-x) = 1 - \sigma(x)$.

В некоторых приложениях не требуются выходные значения, которые можно интерпретировать как вероятности. В таких случаях исследователи могут предпочитать функции активации, выходные значения которых выходят за пределы 0 и 1. Хорошей альтернативой является функция $\tanh(x)$. График этой функции очень похож на график логистической функции, но её выходные значения варьируются от -1 до 1 .

Функция softmax

Логистическая функция позволяет классифицировать признаки по одной из двух категорий. Но многие задачи машинного обучения, такие как классификация ирисов, требуют классификации признаков по нескольким категориям. По этой причине нейронная сеть будет иметь несколько узлов, выходные данные которых объединяются и подаются на вход функции *softmax*.

Функция softmax расширяет логистическую функцию для поддержки нескольких категорий. Сети нужен один узел для каждой категории, а выход softmax определяет вероятность того, что входные данные принадлежат к i -й категории. Математически выход определяется следующим уравнением:

$$\sigma(x_i) = \frac{e^{x_i}}{\sum_{k=1}^N e^{x_k}}$$

В этом уравнении i обозначает категорию, а N — количество категорий. Для классификации ирисов существует три категории, поэтому N будет установлено равным 3.

Слои и глубокое обучение

Отдельные узлы слишком примитивны, чтобы быть полезными на практике, но в сочетании в сети они образуют сложные инструменты для машинного обучения. Взаимосвязанный набор узлов называется *нейронной сетью*, и в этом разделе рассматривается их структура.

Слои

Узлы в нейронной сети организованы в столбцы, которые называют *слоями*. По этой причине нейронные сети часто называют многослойными перцептронами, или MLP. Каждая практическая нейронная сеть имеет как минимум два слоя, и на рис. 9. показан MLP с четырьмя слоями.

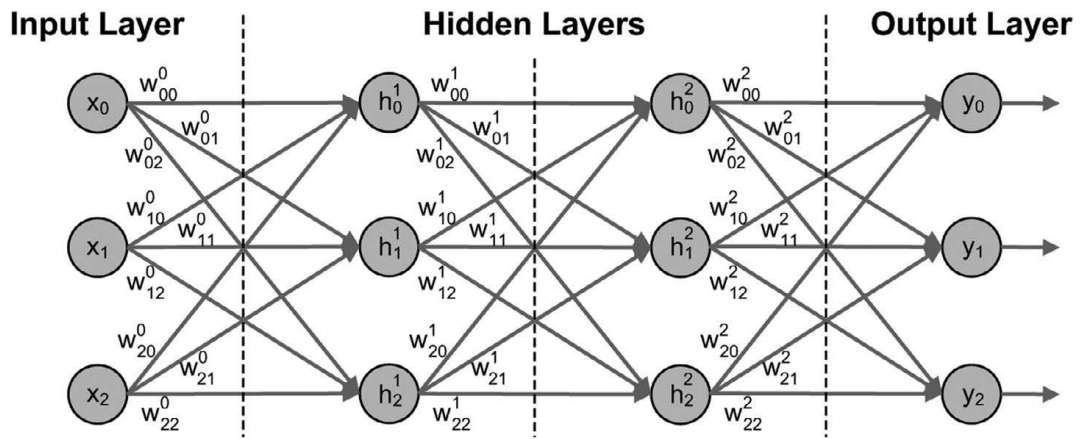


Рис. 9. Четырехслойный перцептрон.

Первый слой — это входной слой, а последний — выходной слой. Промежуточные слои называются *скрытыми слоями*. Слои нумеруются слева направо, входной слой — это слой 0.

Слой является плотным или полносвязным, если каждый из его узлов соединен с каждым узлом в следующем слое. На рис. 9 каждый слой в сети является плотным.

Узлы в скрытом слое обозначаются h_y^x , где x — индекс слоя, а y — индекс узла в слое. Например, h_1^2 обозначает второй узел (1) в третьем (2) слое.

Выход каждого узла вычисляется путем суммирования взвешенных входных данных и передачи суммы в функцию активации. Для простоты будем обозначать выход узла его именем. Например, если функция активации узла h_1^2 — ReLU, его выход получается путем передачи суммы его взвешенных входных сигналов в функцию ReLU. Это можно выразить следующим уравнением:

$$h_1^2 = \text{ReLU}(w_{01}^1 h_0^1 + w_{11}^1 h_1^1 + w_{21}^1 h_2^1)$$

Как показано на рисунке, каждый вес идентифицируется тремя индексами. Обозначая вес как w_{yz}^x , его положение можно определить следующим образом:

- x обозначает слой, содержащий узел, генерирующий взвешенный сигнал
- y обозначает индекс узла, генерирующего взвешенный сигнал
- z обозначает индекс узла, принимающего сигнал

Например, w_{01}^2 обозначает вес в третьем слое (слой 2). Сеть умножает вес на сигнал, выходящий из первого узла (0) и входящий во второй узел (1).

Глубокое обучение

По мере добавления новых скрытых слоев сеть становится более способной к сложным задачам обнаружения и классификации. Когда приложение использует нейронную сеть с несколькими скрытыми слоями, говорят, что оно использует глубокое обучение.

Но есть два основных недостатка добавления большего количества слоев к сети. Во-первых, каждый скрытый слой увеличивает время, необходимое для обучения сети. Во-вторых, каждый новый слой увеличивает вероятность переобучения, что означает, что сеть хорошо справляется с классификацией/прогнозированием в отдельных случаях, но плохо работает с общими данными.

Обучение

Основная сложность в работе с нейронными сетями заключается в обновлении весов таким образом, чтобы узлы в последнем слое выдавали подходящие результаты. Этот процесс называется обучением. Как только нейронная сеть будет полностью обучена, она сможет выполнять задачи классификации (На этом изображении изображена кошка или собака?) или прогнозирования (Какова будет средняя температура завтра?).

Чтобы понять процесс обучения, полезно представить нейронную сеть как многоступенчатую математическую функцию, включающую взвешенные входные данные. Рассмотрим сеть, изображенную на рис. 10.

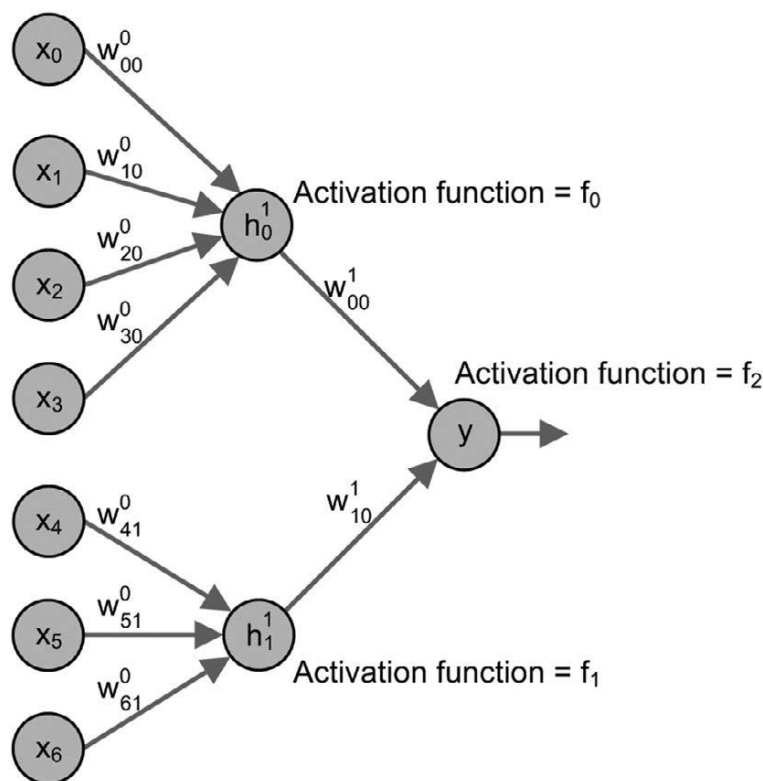


Рис. 10. Пример нейронной сети.

Обозначая функции активации как f_0, f_1 и f_2 , нейронная сеть на рис. 10 воплощает следующее математическое соотношение:

$$y(x_i) = f_2 \left(w_{00}^1 f_0 \left(w_{00}^0 x_0 + w_{10}^0 x_1 + w_{20}^0 x_2 + w_{30}^0 x_3 \right) + w_{10}^1 f_1 \left(w_{41}^0 x_4 + w_{51}^0 x_5 + w_{61}^0 x_6 \right) \right)$$

Цель обучения — вычислить набор весов w_i , которые максимально приближают $y(x_i)$ к наблюдаемым данным. Наиболее популярный метод обучения нейронной сети называется обратным распространением ошибки, который состоит из пяти шагов:

1. Сгенерируйте начальные значения весов w_i .
2. Для каждой партии входных векторов x_i вычислите выходные векторы $y(x_i)$. Этот шаг называется прямым распространением.
3. Вычислите разницу между выходными значениями $y(x_i)$ и наблюдаемыми результатами obs_i . Это называется функцией потерь.
4. Используйте метод оптимизации для обновления весов w_i на основе вычисленной функции потерь.

5. Повторяйте шаги 2–4 до тех пор, пока функция потерь не снизится до приемлемого значения.

Если эти шаги выполнены правильно, нейронная сеть (теоретически) даст приемлемые результаты для реального прогнозирования или классификации. Чтобы прояснить, как можно выполнить эти шаги, далее рассматриваются инициализация весов, вычисление функции потерь и методы оптимизации.

Инициализация весов

Раньше веса всегда инициализировались значением 0. Но исследователи обнаружили, что лучше использовать случайные значения. Будем использовать метод инициализации весов, который учитывает количество входных сигналов, поступающих в узел. Каждому весу присваивается значение из нормального распределения, среднее значение которого равно нулю, а стандартное отклонение σ получается следующим образом:

$$\sigma = \sqrt{\frac{2}{fan\ in}}$$

В этом уравнении *fan in* обозначает количество входных сигналов, поступающих в узел. По мере увеличения количества входных данных стандартное отклонение уменьшается, а начальные значения приближаются к нулю.

Поясним на примере, как это работает. Если у узла четыре входа, σ будет равно $\sqrt{2/4} = 0.707$. Среднее значение равно нулю, поэтому при генерации нормально распределенных весов 68% из них должны находиться в диапазоне от -0.707 до 0.707 . Девяносто пять процентов сгенерированных весов должны иметь значения в диапазоне от -1.414 до 1.414 .

JavaScript не предоставляет функций для генерации нормально распределенных значений, поэтому инициализируем веса с помощью преобразования Бокса-Мюллера. Оно принимает два равномерно распределенных случайных числа, x_1 и x_2 , и генерирует нормально распределенные веса w_1 и w_2 , используя следующие соотношения:

$$w_1 = \sqrt{-2\ln(x_1)} \cos(2\pi x_2) \sigma$$
$$w_2 = \sqrt{-2\ln(x_1)} \sin(2\pi x_2) \sigma$$

В этих уравнениях σ — стандартное отклонение.

Вычисление функции потерь

Во время обучения приложению необходимо измерить, насколько хорошо работает нейронная сеть. Для этого вычисляется разница между значениями выходных узлов сети и наблюдаемыми значениями. Эта разница называется функцией потерь, и метод вычисления функции потерь зависит от многих факторов, включая характер выходных данных нейронной сети.

В начале обучения все веса имеют случайные значения, поэтому выходные данные, вероятно, будут сильно отличаться от желаемых значений (большое значение функции потерь). По мере обучения веса будут обновляться, чтобы привести выходные данные сети в соответствие с наблюдаемыми значениями. Когда выходные данные сети становятся приемлемыми, обучение останавливается (малое значение функции потерь). Функция потерь всегда больше или равна 0, и чем меньше значение функции потерь, тем лучше обучение.

Если нейронная сеть обучается делать прогнозы, функция потерь обычно определяется путем вычисления среднеквадратической ошибки (MSE). Если нейронная сеть обучается классификации признаков, функция потерь определяется путем вычисления отрицательного логарифма правдоподобия. Рассмотрим оба метода.

Прогнозирование и среднеквадратическая ошибка (MSE)

Предположим, вы обучаете нейронную сеть прогнозировать три значения: среднесуточную температуру, среднесуточное барометрическое давление и среднесуточное количество осадков. В этом случае сеть будет иметь три выходных узла — по одному для каждого прогнозируемого значения. Обозначим эти значения y_i (y_0 — прогнозируемая температура, y_1 — прогнозируемое давление, а y_2 — прогнозируемые осадки).

Во время обучения приложение будет сравнивать y_i с наблюдаемыми значениями за предыдущие дни. Чтобы оценить эффективность обучения, приложение измеряет разницу между прогнозируемыми и наблюдаемыми значениями.

Распространенный способ измерения этой разницы (функции потерь) включает вычисление среднеквадратической ошибки, или MSE. При этом каждое прогнозируемое значение (y_i) вычитается из каждого наблюдаемого значения (obs_i), результат возводится в квадрат, а квадраты разностей суммируются. Если имеется N прогнозируемых значений, MSE вычисляет потери по следующей формуле:

$$loss = \frac{1}{N} \sum_{i=0}^{N-1} (obs_i - y(x_i))^2$$

Рассмотрим пример. Предположим, что средняя температура вчера составляла 14.5°C, среднее барометрическое давление — 760.8 мм рт. ст., а среднее количество осадков — 682.0 мм. Составим вектор [14.5, 760.8, 682.0]. Теперь предположим, что выходные данные нейронной сети равны [12.8, 772.4, 689.4]. Функцию потерь можно вычислить следующим образом:

$$\begin{aligned} loss &= \frac{1}{N} \sum_{i=0}^{N-1} (obs_i - y(x_i))^2 = \\ &= \frac{1}{3} [(14.5 - 12.8)^2 + (760.8 - 772.4)^2 + (682.0 - 689.4)^2] = \\ &= \frac{1}{3} (2.89 + 134.56 + 54.76) = \\ &= 64.07 \end{aligned}$$

Поскольку этот метод складывает квадраты разностей, результат всегда больше или равен нулю. По этой причине MSE является приемлемым методом вычисления функции потерь в машинном обучении.

Классификация и метод максимального правдоподобия

Теперь рассмотрим нейронные сети, предназначенные для классификации признаков, а не для прогнозирования. Например, предположим, что цель нейронной сети — определить, является ли животное на изображении кошкой, собакой или лошадью (и это всегда одна из этих трех категорий).

В этом случае выходной слой сети будет иметь три узла, и каждый будет выдавать вероятность, связанную с соответствующей категорией. y_0 — это вычисленная вероятность того, что

животное на изображении — кошка, y_1 — это вычисленная вероятность того, что животное на изображении — собака, и y_2 — это вычисленная вероятность того, что животное на изображении — лошадь. Поскольку мы имеем дело с вероятностями, каждое значение будет находиться в диапазоне от 0.0 до 1.0, и сумма всегда должна быть близка к 1.0.

В идеале нейронная сеть должна выдавать на выходе 1.0 для правильной категории и 0.0 для неправильной категории. На практике этого никогда не происходит.

Если задействованы только две категории, наиболее распространенной функцией активации является логистическая функция, обсуждавшаяся ранее. Для более чем двух категорий вместо нее используется функция softmax.

Для вычисления потерь нейронные сети классификации используют метод кросс-энтропии, также называемый логарифмической функцией правдоподобия. Если наблюдаемые значения — obs_i , а предсказанные значения — $y(x_i)$, потери вычисляются следующим образом:

$$loss = -\sum_{i=0}^{N-1} obs_i \cdot \ln(y(x_i)) \quad (1)$$

На первый взгляд может показаться, что это даст отрицательные значения потерь. Но важно помнить, что все значения являются вероятностями, то есть они находятся в диапазоне от 0.0 до 1.0. Логарифм вероятности всегда отрицателен или равен нулю, поэтому эти потери всегда будут положительными или равны нулю.

Рассмотрим пример. Предположим, что сеть выдает три вероятности: вероятность того, что ирис принадлежит к виду *setosa*, вероятность того, что он принадлежит к виду *versicolor*, и вероятность того, что он принадлежит к виду *virginica*. Если заранее известно, что ирис относится к виду *versicolor*, то наблюдаемые значения obs_i равны [0,0, 1,0, 0,0].

Конечно, нейронная сеть не будет выдавать эти идеальные значения, поэтому предположим, что она выдает вектор вероятностей [0.09, 0.76, 0.15]. Функция потерь может быть вычислена следующим образом:

$$loss = -[0.0 \cdot \log(0.09) + 1.0 \cdot \ln(0.76) + 0.0 \cdot \ln(0.15)] = -\ln(0.76) = 0.2744$$

Таким образом, только один из членов в функции потерь будет ненулевым. Не имеет значения, какую вероятность сеть присваивает неправильным классам. Единственная значимая вероятность — это вероятность, присвоенная правильному классу.

Вычисление функции потерь важно, но для улучшения работы сети нам нужен способ обновления ее весов таким образом, чтобы уменьшить функцию потерь. Эта процедура называется оптимизацией.

Оптимизация методом градиентного спуска

Метод градиентного спуска — это старейший алгоритм оптимизации, используемый в машинном обучении. Данный метод является простым, но медленным. Он игнорирует предыдущие значения, что может приводить к осцилляциям.

Оптимизация методом градиентного спуска основана на важном математическом факте: функция убывает быстрее всего в точке P в направлении, определяемом отрицательным градиентом в точке P.

Алгоритм градиентного спуска вычисляет градиент функции потерь и обновляет переменные модели до тех пор, пока градиент функции потерь не приблизится к нулю. Для математического выражения этого необходимо ввести некоторые обозначения:

- Набор обучаемых переменных (весов) обозначим θ . Значения весов на шаге t обозначаются θ_t .
- Функция потерь, являющаяся функцией переменных модели, обозначается $J(\theta)$. Градиент функции потерь обозначим $\nabla J(\theta)$.
- Скорость обучения, обозначим η — это единственное значение, которое влияет на то, насколько будет обновляться θ_j от шага к шагу.

С учетом этих обозначений каждый шаг метода оптимизации градиентного спуска можно выразить следующим уравнением:

$$\theta_t = \theta_{t-1} - \eta \nabla J(\theta)$$

Рассмотрим простой пример. Предположим, что старое значение веса w_0 равно 0.4, а частная производная функции потерь по w_0 равна -0.5 . Функция потерь увеличивается по мере уменьшения w_0 , поэтому мы хотим увеличить следующее значение w_0 . Первый элемент θ_{t-1} равен 0.4, поэтому это уравнение устанавливает θ_t (следующее значение w_0) равным $0.4 - \eta(-0.5)$ или $0.4 + 0.5\eta$. Аналогичная операция выполняется для других весов в θ_t .

Теперь предположим, что частная производная функции потерь по w_0 равна нулю. В этом случае функция потерь вообще не зависит от w_0 , поэтому уравнение оставляет значение w_0 неизменным.

По мере продолжения обучения элементы $\nabla J(\theta)$ должны приближаться к минимуму, что означает, что каждый новый набор переменных должен быть приблизительно равен предыдущему набору. На этом этапе оптимизация завершается, поскольку она (теоретически) нашла набор весов, обеспечивающих минимальные потери. Это условие называется сходимостью.

Значение η выбирается разработчиком, и выбор этого значения является важным. Если η слишком велико, алгоритм будет продвигаться быстро, но он может обходить минимум и никогда не достичь конечного значения.

Если η слишком мало, алгоритм будет двигаться точнее, но это займет много времени. Кроме того, алгоритм может остановиться в локальном минимуме вместо глобального. Рис. 12 иллюстрирует разницу.

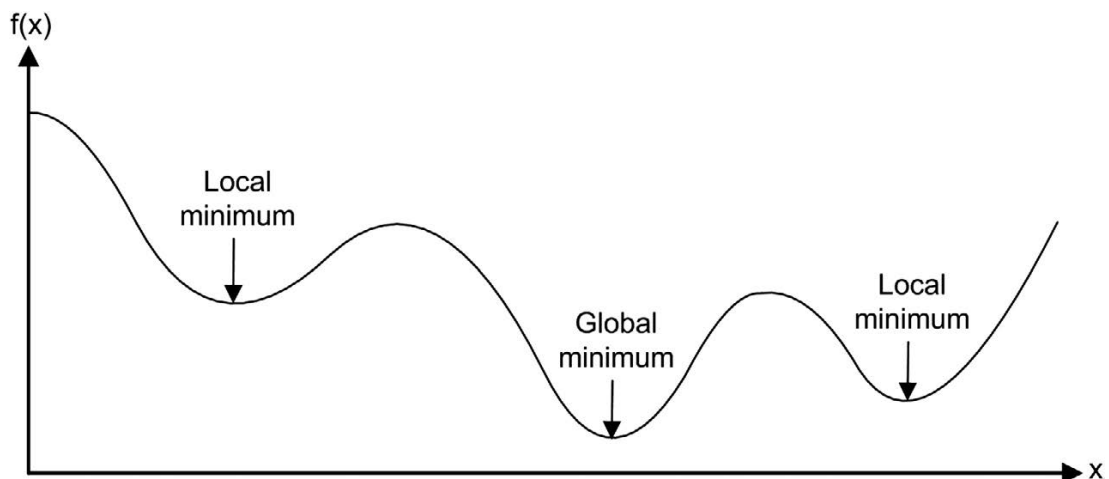


Рис. 12. Локальный и глобальный минимумы.

Помимо сложности нахождения глобального минимума, метод градиентного спуска имеет три существенных недостатка:

- Он, как правило, медленно сходится к минимальному значению.
- Он может оптимизировать только дифференцируемые функции.
- Он может колебаться между значениями и никогда не достигать минимума.

Последний аспект заслуживает объяснения. Если скорость обучения высока, алгоритм может скакать между парой точек, никогда не достигая минимума. Это скачкообразное движение называется осцилляцией. Приложения могут уменьшить вероятность осцилляции, снизив скорость обучения, но тогда обучение занимает больше времени.

Мини-пакетная обработка

При обучении нейронной сети одной из основных проблем является частота обновления весов. С одной стороны, приложение может вычислить потери для всех входных векторов, а затем использовать среднее значение функции потерь для обновления весов. Это называется пакетной обработкой. С другой стороны, приложение может обновлять веса после вычисления потери для каждого отдельного входного вектора.

Будем использовать мини-пакетную обработку, которая служит компромиссом между этими двумя крайностями. Разделим обучающий набор на группы и будем обновлять веса после вычисления средней ошибки для каждой группы векторов.

В приложении IrisNetwork имеется в общей сложности 150 векторов, содержащих размеры ирисов и связанные с ними классификации. Приложение использует 144 вектора для обучения, а оставшиеся 6 — для тестирования обученной нейронной сети. На рис. 1 показаны примеры результатов шести тестов.

В процессе обучения приложение разбивает 144 обучающих вектора на девять мини-пакетов по 16 векторов в каждом. В каждой группе 16 векторов обрабатываются параллельно, и для каждого мини-пакета вычисляется средний вектор градиента функции потерь. Это значение используется для обновления весов.

Пример применения — классификация ирисов

Приложение IrisNetwork реализует базовую нейронную сеть с помощью вычислительного шейдера WebGPU. Приложение передаёт параметры ирисов в нейронную сеть, которая выдает три значения на выходе: вероятность того, что ирис принадлежит виду *setosa*, вероятность того, что он принадлежит виду *versicolor*, и вероятность того, что он принадлежит виду *virginica*. На рис.13 показана структура сети.

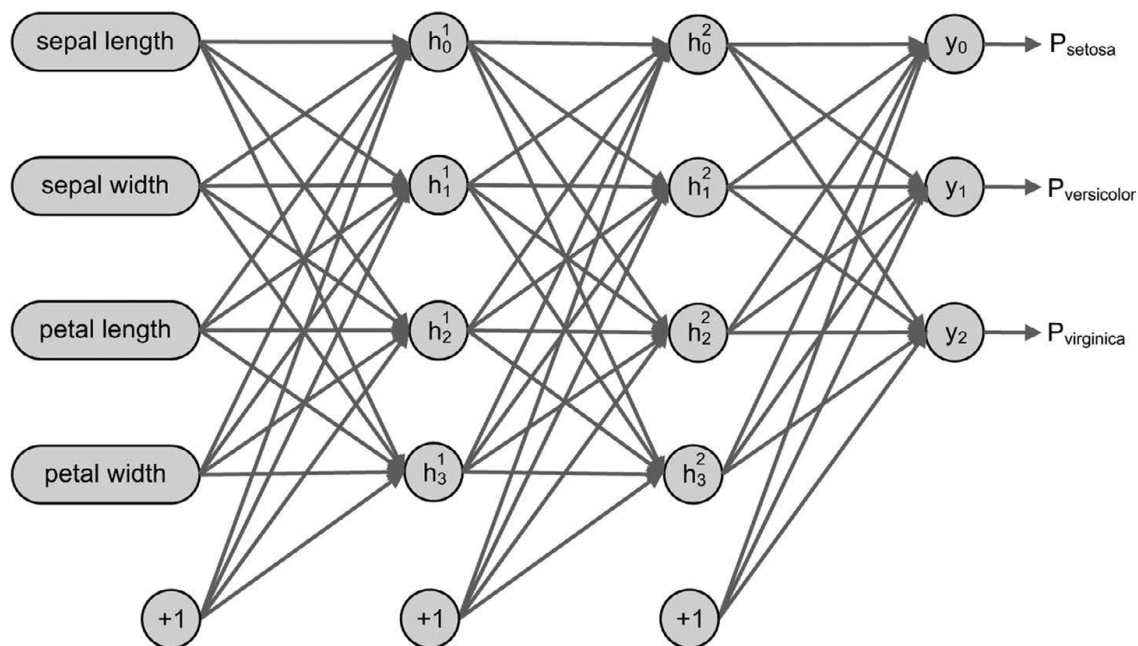


Рис. 13. Структура нейронной сети для классификации ирисов.

Сеть имеет два скрытых слоя по четыре узла каждый и выходной слой с тремя узлами. Каждый из одиннадцати узлов получает пять взвешенных входных сигналов: четыре от входных сигналов и один от смещения, обозначенного как +1. Таким образом, сеть имеет в общей сложности $11 * 4 = 44$ обычных веса и 11 весов смещения.

Функция активации узлов в скрытых слоях — ReLU, что означает, что каждый скрытый узел будет выдавать максимум, равный нулю, или сумму своих взвешенных входных сигналов. Функция активации узлов в последнем слое — softmax, поэтому выходные узлы будут выдавать вероятности, соответствующие оценке классификации ирисов.

IrisNetwork создает нейронную сеть и предоставляет ей данные. В табл. перечислены буферы хранения, в которых содержатся эти данные.

Буфер	Описание
train_data	Содержит характеристики ирисов, используемые для обучения
train_results	Содержит правильные классификации ирисов
weights	Веса, применяемые к соединениям между узлами
bias_weights	Веса смещений, применяемые к соединениям между узлами
test_data	Содержит характеристики ирисов, используемые для тестирования
test_results	Хранит результаты классификации нейронной сети

После определения буферов хранения шейдер объявляет 12 `override` переменных. В таблице перечислены каждая из них и значения, предоставленные приложением.

Переменная	Значение	Описание
group_size	64	Количество вызовов на группу
batch_size	16	Количество обучающих векторов, обрабатываемых за итерацию
num_train_points	144	Количество обучающих векторов (каждый вектор содержит длину чашелистика, ширину чашелистика, длину лепестка и ширину лепестка)
num_test_points	6	Количество тестовых векторов (каждый вектор содержит длину чашелистика, ширину чашелистика, длину лепестка и ширину лепестка)

num_inputs	4	Количество входных данных в первом слое сети (без учета смещения)
l1_size	4	Количество узлов в первом слое
l2_size	4	Количество узлов во втором слое
l3_size	3	Количество узлов в выходном слое
num_weights	44	Количество обычных весов
num_bias	11	Количество весов смещений
num_epochs	150	Количество циклов обучения
eta	0.001	Скорость обучения, используемая во время оптимизации

После объявления этих переменных шейдер определяет переменные в памяти рабочей группы. В таблице перечислены каждая из этих переменных и её содержимое.

Переменная рабочей группы	Описание
layer1	Хранит выходные данные каждого узла в слое 1
layer2	Хранит выходные данные каждого узла в слое 2
layer3	Хранит выходные данные каждого узла в слое 3
J	Хранит производные функции потерь от весов сети
J_bias	Хранит производные функции потерь от весов смещения

Обучение сети

Для объяснения принципа работы вычислительного шейдера разделим его работу на две части: обучение и тестирование.

Большая часть кода в вычислительном шейдере используется для обучения сети, которое включает в себя обновление весов до тех пор, пока сеть не начнет выдавать приемлемые выходные значения. Обучение — это длительный процесс, требующий двух циклов выполнения. Внешний цикл выполняется 150 раз. Внутренний цикл выполняется девять раз для каждого мини-пакета.

Обработка слоев с помощью мини-пакетов

Вычислительный шейдер делит обучающие векторы на 16 мини-пакетов в процессе обучения. Каждый мини-пакет обрабатывается четырьмя вызовами, и каждый вызов использует свой глобальный идентификатор для вычисления `batch_id`, который принимает значения от 0 до 15, и `node_id`, который принимает значения от 0 до 3. Вызов использует свой `batch_id` для чтения обучающих данных и `node_id` для определения узла сети за расчет которого он отвечает.

Следующий код показывает, как узлы сети обрабатываются вызовами шейдера. Поскольку сеть имеет три слоя, код разделен на три части.

```
// First layer
sum = 0.0;
for (i = 0; i < num_inputs; i++) {
    sum += train_data[train_id + i] * weights[weight_addr + i];
}
sum += bias_weights[bias_addr];
layer1[l1_id + node_id] = max(0.0, sum);
weight_addr += l1_size * num_inputs;
bias_addr += num_inputs;
workgroupBarrier();
```

```

// Second layer
sum = 0.0;
for (i = 0; i < l1_size; i++) {
    sum += layer1[l1_id + i] * weights[weight_addr + i];
}
sum += bias_weights[bias_addr];
layer2[l2_id + node_id] = max(0.0, sum);
weight_addr += l2_size * l1_size;
bias_addr += l2_size;
workgroupBarrier();
// Third layer
if(node_id < l3_size) {
    sum = 0.0;
    for (i = 0; i < l2_size; i++) {
        sum += layer2[l2_id + i] * weights[weight_addr + i];
    }
    sum += bias_weights[bias_addr];
    layer3[l3_id + node_id] = sum;
}
workgroupBarrier();

```

При запуске шейдера каждый вызов считывает вектор, содержащий четыре элемента, из буфера `train_data`. Затем, во время обработки слоя 1, он умножает каждый входной сигнал на вес из буфера весов. Затем он складывает взвешенные входные данные и добавляет вес смещения. Если сумма больше нуля, сумма будет сохранена. В противном случае будет сохранен ноль.

Обработка слоев 2 и 3 аналогична. В каждом случае вызов выполняет обработку узла на основе его `node_id`. Он считывает входные данные из предыдущего слоя, умножает их на веса и добавляет вес смещения.

После каждого этапа шейдер выполняет `workgroupBarrier`, который заставляет вызовы останавливаться до тех пор, пока все вызовы рабочей группы не завершат работы. Таким образом, все вычисления одного слоя должны завершиться до того, как сможет начаться следующий слой.

Вычисление градиента функции потерь — Часть 1

Узлы в слое 3 используют функцию активации softmax вместо ReLU, поэтому эти узлы не сравнивают сумму своих взвешенных входных данных с нулем. Вместо этого вычислительный шейдер выполняет следующий код:

```

// Compute the softmax outputs
d = exp(layer3[l3_id]) + exp(layer3[l3_id+1]) + exp(layer3[l3_id+2]);

```

```

sm1 = exp(layer3[l3_id])/d;
sm2 = exp(layer3[l3_id+1])/d;
sm3 = exp(layer3[l3_id+2])/d;
// Compute loss gradients for output nodes
var res = train_results[iter * batch_size + batch_id];
layer3[l3_id] = sm1 - res[0];
layer3[l3_id+1] = sm2 - res[1];
layer3[l3_id+2] = sm3 - res[2];

```

Первая часть этого кода получает значение softmax из значений третьего слоя. Вторая часть кода считывает правильные значения классификации для обучающего вектора из буфера `train_results`. Затем она вычитает каждый результат из соответствующего выходного значения softmax. Чтобы понять, почему вычислительный шейдер делает это, необходимо понять, как вычисляются градиенты для функции потерь кросс-энтропии.

Выход нейронной сети представляет собой вектор p_i , содержащий три значения:

- p_0 — вероятность того, что ирис принадлежит к виду *setosa*;
- p_1 — вероятность того, что ирис принадлежит к виду *versicolor*;
- p_2 — вероятность того, что ирис принадлежит к виду *virginica*.

Если в векторе obs_i содержатся правильные вероятности, то кросс-энтропийная функция потерь для обучающей выборки вычисляется по формуле (1).

Для вычисления градиента этой функции вычислительный шейдер должен найти производную логарифма кросс-энтропийной функции потерь. Производная $\log(x)$ равна $1/x$. Если y_i — сумма взвешенных входных данных, поступающих в выходной узел, то производная функции потерь по y_i определяется следующим образом:

$$\frac{\partial J}{\partial y_j} = -\sum_{i=0}^2 obs_i \cdot \frac{\partial}{\partial y_j} \ln(p_i) = -\sum_{i=0}^2 obs_i \cdot \frac{\partial \ln(p_i)}{\partial p_i} \frac{\partial p_i}{\partial y_j} = -\sum_{i=0}^2 obs_i \cdot \frac{1}{p_i} \frac{\partial p_i}{\partial y_j}$$

Вероятности p_i вычисляются с помощью функции softmax, поэтому для нахождения градиента функции потерь первым шагом является вычисление производной функции softmax по ее входному значению y_i . Если $i = j$ то получаем следующее выражение:

$$\begin{aligned}
\frac{\partial p_i}{\partial y_j} &= \frac{\partial}{\partial y_j} \left(\frac{e^{y_i}}{\sum_k e^{y_k}} \right) = \frac{\left(\frac{\partial}{\partial y_j} e^{y_i} \right) \sum_k e^{y_k} - e^{y_i} \left(\frac{\partial}{\partial y_j} \sum_k e^{y_k} \right)}{\left(\sum_k e^{y_k} \right)^2} = \\
&= \frac{e^{y_i} \sum_k e^{y_k} - e^{y_i} e^{y_j}}{\left(\sum_k e^{y_k} \right)^2} = \frac{e^{y_i} \left(\sum_k e^{y_k} - e^{y_j} \right)}{\left(\sum_k e^{y_k} \right)^2} = \\
&= \frac{e^{y_i}}{\sum_k e^{y_k}} \left(\frac{\sum_k e^{y_k} - e^{y_j}}{\sum_k e^{y_k}} \right) = p_i (1 - p_j)
\end{aligned}$$

Если $i \neq j$, то производная функции softmax вычисляется следующим образом:

$$\frac{\partial p_i}{\partial y_j} = \frac{0 - e^{y_i} e^{y_j}}{\left(\sum_k e^{y_k} \right)^2} = \frac{-e^{y_i}}{\sum_k e^{y_k}} \frac{e^{y_j}}{\sum_k e^{y_k}} = -p_i p_j$$

Для формирования единого выражения, учитывающего оба условия, используем дельту-функцию Кронекера, δ_{ij} , которая определяется следующим образом:

$$\delta_{ij} = \begin{cases} 0, & \text{если } i \neq j \\ 1, & \text{если } i = j \end{cases}$$

Используя это значение, мы можем объединить оба выражения для производной softmax в одно уравнение:

$$\frac{\partial p_i}{\partial y_j} = p_i (\delta_{ij} - p_j)$$

Подставив это в выражение для градиента функции потерь, мы получаем следующий результат:

$$\begin{aligned}
\frac{\partial J}{\partial y_j} &= -\sum_{i=0}^2 obs_i \frac{1}{p_i} \frac{\partial p_i}{\partial y_j} = -\sum_{i=0}^2 obs_i \frac{1}{p_i} [p_i (\delta_{ij} - p_j)] = -\sum_{i=0}^2 obs_i (\delta_{ij} - p_j) = \\
&= -obs_j (1 - p_j) + \sum_{i \neq j} obs_i p_j = -obs_j + obs_j p_j + \sum_{i \neq j} obs_i p_j = \\
&= p_j \left(obs_j + \sum_{i \neq j} obs_i \right) - obs_j = p_j - obs_j
\end{aligned}$$

Последняя строка получается потому, что сумма obs_i равна 1. Действительно, obs_i содержит известные вероятности вида ириса — одно из трех значений всегда будет равно 1, а остальные — 0. Таким образом, вычислительный шейдер действительно должен вычитать правильные значения из выходных данных softmax.

Вычисление градиента функции потерь — Часть 2

Нахождение производной функции потерь кросс-энтропии по y_j (входные данные функции softmax) является важным шагом, но для получения окончательного градиента потерь вычислительному шейдеру необходимо продолжить расчет. Цель состоит в том, чтобы получить вектор, содержащий частную производную функции потерь по каждому весу w_i :

$$\frac{\partial J}{\partial w_i} = \frac{\partial J}{\partial y_j} \frac{\partial y_j}{\partial w_i}$$

Таким образом, требуется найти второй член, который является частной производной входных данных функции softmax (y_j) по каждому весу (w_i). Чтобы понять, как это можно сделать для рассматриваемой сети, рассмотрим диаграмму, показанную на рис. 14.

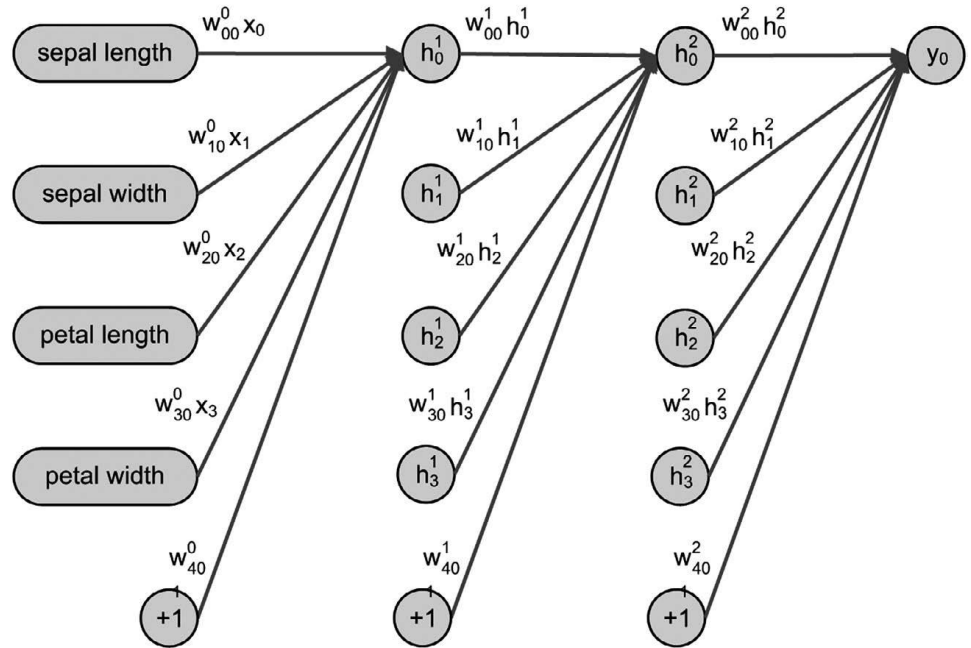


Рис. 14. Связь входных данных функции softmax с весами.

Эта диаграмма иллюстрирует входные данные, сигналы и веса, влияющие на самый верхний выходной узел, y_0 . Как обсуждалось ранее, каждый вес обозначается w^x_{yz} , а его индексы можно интерпретировать следующим образом:

- x обозначает слой, содержащий узел, генерирующий взвешенный сигнал
- y обозначает индекс узла, генерирующего сигнал, который необходимо взвесить
- z обозначает индекс узла, принимающего сигнал

Предположим, мы хотим узнать частную производную y_0 по w^2_{00} . Для этого рассмотрим выражение:

$$y_0 = w^2_{00}h^2_0 + w^2_{10}h^2_1 + w^2_{20}h^2_2 + w^2_{30}h^2_3 + w^2_{40}$$

$$\frac{\partial y_0}{\partial w^2_{00}} = h^2_0$$

Теперь рассмотрим расчет частной производной y_0 по w^1_{00} .

Скрытые узлы используют функцию активации ReLU. Если x больше 0, $\text{ReLU}(x)$ возвращает x и её производная равна 1. Если x меньше или равно 0, ReLU возвращает 0 и её производная равна 0. Тогда:

$$\begin{aligned}
 y_0 &= w_{00}^2 h_0^2 + w_{10}^2 h_1^2 + w_{20}^2 h_2^2 + w_{30}^2 h_3^2 + w_{40}^2 \\
 \frac{\partial y_0}{\partial w_{00}^1} &= \frac{\partial}{\partial w_{00}^1} [w_{00}^2 h_0^2 + w_{10}^2 h_1^2 + w_{20}^2 h_2^2 + w_{30}^2 h_3^2 + w_{40}^2] = \frac{\partial}{\partial w_{00}^1} (w_{00}^2 h_0^2) = w_{00}^2 \frac{\partial}{\partial w_{00}^1} h_0^2 \\
 w_{00}^2 \frac{\partial}{\partial w_{00}^1} h_0^2 &= w_{00}^2 \frac{\partial}{\partial w_{00}^1} [\text{ReLU}(w_{00}^1 h_0^1 + w_{10}^1 h_1^1 + w_{20}^1 h_2^1 + w_{30}^1 h_3^1 + w_{40}^1)] \\
 \frac{\partial y_0}{\partial w_{00}^1} &= \begin{cases} 0, & \text{если } h_0^2 \leq 0 \\ w_{00}^2 h_0^1, & \text{если } h_0^2 > 0 \end{cases}
 \end{aligned}$$

Мы можем получить и другие производные, используя аналогичный метод. Например, следующее уравнение представляет собой частную производную y_0 по w_{00}^0 .

$$\frac{\partial y_0}{\partial w_{00}^0} = \begin{cases} 0, & \text{если } h_0^2 \leq 0 \text{ или } h_0^1 \leq 0 \\ w_{00}^2 w_{00}^1 x_0, & \text{если } h_0^2 > 0 \text{ и } h_0^1 > 0 \end{cases}$$

Производные функции потерь проще вычислить для весов, ближайших к выходным узлам. По этой причине вычислительный шейдер вычисляет производные функции потерь, выполняя серию циклов, которые идут от слоя 3 к слою 1. Производные функции потерь от весов узлов хранятся в переменной J в памяти рабочей группы. Производные функции потерь от весов узлов смещения хранятся в переменной J_bias .

После вычисления градиентов потерь вычислительный шейдер обновляет веса и смещения сети с помощью следующего кода:

```

// Update weights and biases
if (id.x == 0) {
    // Iterate through weights
    for (i = 0; i < num_weights; i++) {
        sum = 0.0;
        for (j = 0; j < batch_size; j++) {
            sum += J[j * num_weights + i];
        }
        weights[i] -= eta * sum;
    }
}

```

```

// Iterate through biases
for (i = 0; i < num_bias; i++) {
    sum = 0.0;
    for (j = 0; j < batch_size; j++) {
        sum += J_bias[j * num_bias + i];
    }
    bias_weights[i] -= eta * sum;
}

storageBarrier();

```

Скорость обучения η равна 0.001, поэтому веса не сильно меняются от одной итерации к другой. Но после достаточного количества итераций обучения шейдер устанавливает веса на значения, подходящие для правильной классификации ирисов.

Тестирование сети

После последней итерации обучения вычислительный шейдер пытается классифицировать ирис, характеристики которого хранятся в буфере `test_data`. Для каждого входного вектора вычислительный шейдер выдает три значения: вероятность того, что радужная оболочка глаза принадлежит виду `setosa`, вероятность того, что она принадлежит виду `versicolor`, и вероятность того, что она принадлежит виду `virginica`.

Для выполнения теста шейдер обрабатывает каждый узел сети. Следующий код показывает, как это делается:

```

// Iterate through test points
for (var test: u32 = 0; test < num_test_points; test++) {
    var test_addr = test * num_inputs;
    // First layer
    if (batch_id == 0) {
        sum = 0.0;
        for (i = 0; i < num_inputs; i++) {
            sum += test_data[test_addr + i] *
                weights[node_id * num_inputs + i];
        }
        sum += bias_weights[node_id];
        layer1[node_id] = max(0.0, sum);
    }
    workgroupBarrier();
    // Second layer

```



```

weight_addr = l1_size * num_inputs;
bias_addr = l1_size;
if (batch_id == 0) {
    sum = 0.0;
    for (i = 0; i < l1_size; i++) {
        sum += layer1[i] *
            weights[weight_addr + node_id * l1_size + i];
    }
    sum += bias_weights[bias_addr + node_id];
    layer2[node_id] = max(0.0, sum);
}
workgroupBarrier();
// Third layer
weight_addr += l2_size * l1_size;
bias_addr += l2_size;
if((batch_id == 0) && (node_id < l3_size)) {
    sum = 0.0;
    for (i = 0; i < l2_size; i++) {
        sum += layer2[i] *
            weights[weight_addr + node_id * l2_size + i];
    }
    sum += bias_weights[bias_addr + node_id];
    layer3[node_id] = sum;
}
workgroupBarrier();
// Update test result buffer
If (id.x == 0) {
    d = exp(layer3[0]) + exp(layer3[1]) + exp(layer3[2]);
    test_results[test * 3] = exp(layer3[0])/d;
    test_results[test * 3 + 1] = exp(layer3[1])/d;
    test_results[test * 3 + 2] = exp(layer3[2])/d;
}
storageBarrier();
}

```

Узлы в слое 1 считывают входные тестовые данные, умножают их на веса и добавляют к сумме вес смещения. Узлы в слое 2 получают взвешенные выходные данные узлов слоя 1, а узлы в слое 3 получают взвешенные выходные данные узлов слоя 2. Итоговые вероятности вычисляются путем вычисления функции softmax для значений слоя 3.

Задание для самостоятельной работы

1. Поварьируйте параметром `num_epochs` (количество циклов обучения). Как меняются результаты классификации?
2. Добавьте в нейронную сеть еще один скрытый слой. Проведите сравнение результатов классификации двух нейронных сетей.